

Projeto Integrador — FAM 2026.1

TransLog

Sistema de Gestão Logística com Precificação Inteligente

Programação Orientada a Objetos

Prof. Evandro Santaclara

Turma 57-05-B — Ciência da Computação

Integrantes:

Diego Henrique C. Custódio	20240153
Gabriela Andrade Brito	20240868
Leonardo Dos Santos	20221957
Pedro Henrique A. Marcandali	20241595
Vinícius Eduardo da Silva	20240289

Americana, 17 de maio de 2026.

1. Diagrama de Classes

O sistema TransLog é modelado em 8 classes principais (entidades JPA). A classe Viagem é o ponto central do domínio, agregando referências para Veículo, Rota, Carga, Motorista e Cliente. A classe Frete pode opcionalmente referenciar uma Viagem (no caso de cálculos avulsos sem viagem cadastrada, o campo viagem fica nulo).

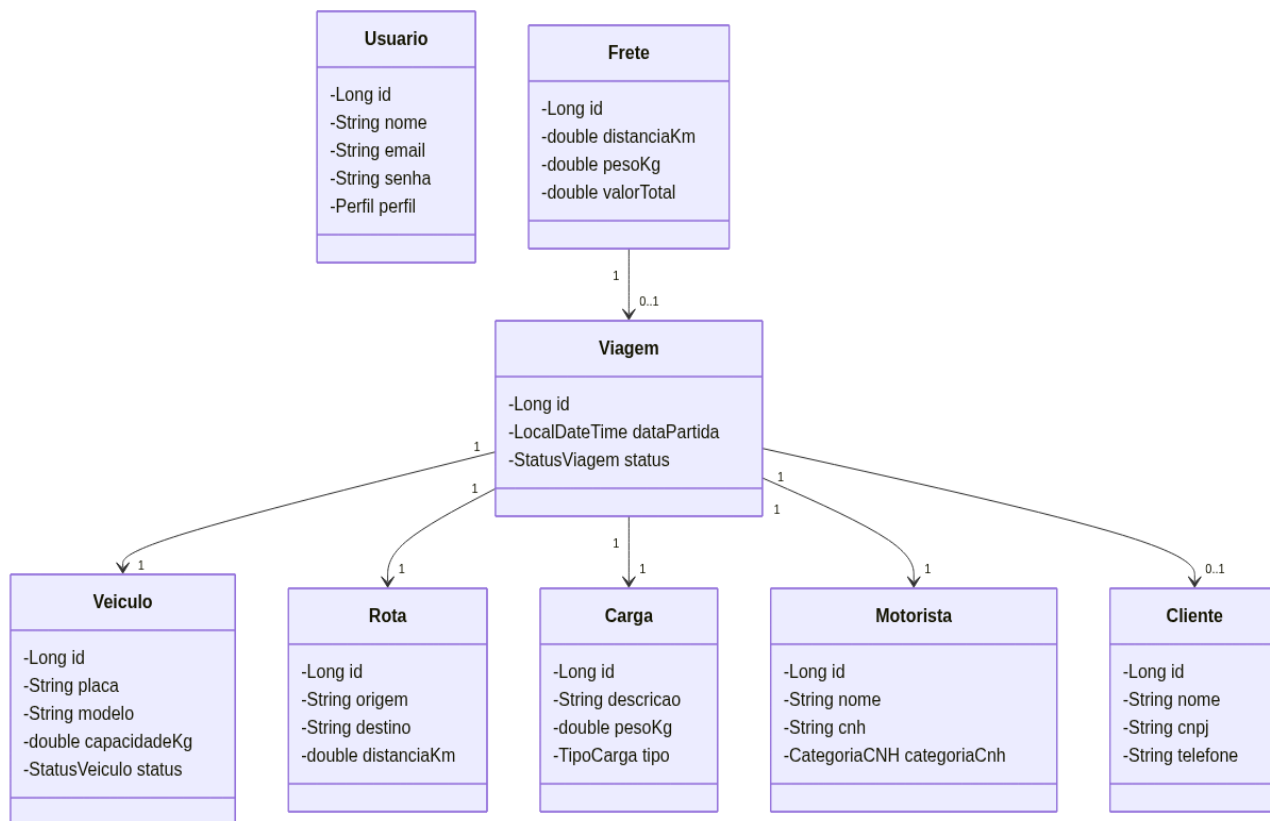


Figura 1 — Diagrama de classes do TransLog (multiplicidades indicadas nas arestas).

Relacionamentos e multiplicidades

Origem	Cardinalidade	Destino	Descrição
Viagem	N : 1	Veículo	Uma viagem usa um veículo; um veículo executa várias ao longo do tempo
Viagem	N : 1	Rota	Cada viagem percorre uma rota cadastrada
Viagem	N : 1	Carga	Cada viagem transporta uma carga
Viagem	N : 1	Motorista	Cada viagem é conduzida por um motorista
Viagem	N : 0..1	Cliente	Cliente opcional (viagens internas podem não ter)
Frete	N : 0..1	Viagem	Cálculo pode ser vinculado a uma viagem ou avulso (simulação)

2. Código Fonte das Classes Principais

A seguir, o código fonte das classes de domínio (Model) e dos componentes que aplicam as regras de negócio (Service e Controller).

2.1 Classes do Modelo (Model)

2.1. Usuario.java

```
package com.transportadora.logistica.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    private String nome;

    @NotBlank
    @Email
    @Column(unique = true)
    private String email;

    @NotBlank
    private String senha;

    @Enumerated(EnumType.STRING)
    private Perfil perfil = Perfil.OPERADOR;

    private boolean ativo = true;

    public enum Perfil {
        ADMIN, OPERADOR, CONSULTA
    }

    public Usuario() {}

    public Usuario(String nome, String email, String senha, Perfil perfil) {
        this.nome = nome;
        this.email = email;
        this.senha = senha;
        this.perfil = perfil;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getSenha() { return senha; }
```

```

    public void setSenha(String senha) { this.senha = senha; }

    public Perfil getPerfil() { return perfil; }

    public void setPerfil(Perfil perfil) { this.perfil = perfil; }

    public boolean isAtivo() { return ativo; }

    public void setAtivo(boolean ativo) { this.ativo = ativo; }
}

```

2.2. *Cliente.java*

```

package com.transportadora.logistica.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;

@Entity
@Table(name = "clientes")
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    private String nome;

    private String telefone;

    private String cidade;

    private String email;

    @Column(unique = true)
    private String cnpj;

    public Cliente() {}

    public Cliente(String nome, String telefone, String cidade) {
        this.nome = nome;
        this.telefone = telefone;
        this.cidade = cidade;
    }

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public String getNome() { return nome; }

    public void setNome(String nome) { this.nome = nome; }

    public String getTelefone() { return telefone; }

    public void setTelefone(String telefone) { this.telefone = telefone; }

    public String getCidade() { return cidade; }

    public void setCidade(String cidade) { this.cidade = cidade; }

    public String getEmail() { return email; }

    public void setEmail(String email) { this.email = email; }

    public String getCnpj() { return cnpj; }

    public void setCnpj(String cnpj) { this.cnpj = cnpj; }
}

```

2.3. Motorista.java

```
package com.transportadora.logistica.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import java.time.LocalDate;

@Entity
@Table(name = "motoristas")
public class Motorista {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    private String nome;

    @NotBlank
    @Column(unique = true)
    private String cnh;

    @Enumerated(EnumType.STRING)
    private CategoriaCNH categoriaCnh;

    private LocalDate validadeCnh;

    private String telefone;

    private boolean ativo = true;

    public enum CategoriaCNH {
        A, B, C, D, E
    }

    public Motorista() {}

    public Motorista(String nome, String cnh, String telefone) {
        this.nome = nome;
        this.cnh = cnh;
        this.telefone = telefone;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getCnh() { return cnh; }
    public void setCnh(String cnh) { this.cnh = cnh; }

    public CategoriaCNH getCategoriaCnh() { return categoriaCnh; }
    public void setCategoriaCnh(CategoriaCNH categoriaCnh) { this.categoriaCnh = categoriaCnh; }

    public LocalDate getValidadeCnh() { return validadeCnh; }
    public void setValidadeCnh(LocalDate validadeCnh) { this.validadeCnh = validadeCnh; }

    public String getTelefone() { return telefone; }
    public void setTelefone(String telefone) { this.telefone = telefone; }

    public boolean isAtivo() { return ativo; }
    public void setAtivo(boolean ativo) { this.ativo = ativo; }
```

```
}
```

2.4. Veiculo.java

```
package com.transportadora.logistica.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

@Entity
@Table(name = "veiculos")
public class Veiculo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Column(unique = true)
    private String placa;

    @NotBlank
    private String modelo;

    @NotBlank
    private String marca;

    private int anoFabricacao;

    @Positive
    private double capacidadeKg;

    @Enumerated(EnumType.STRING)
    private TipoVeiculo tipo;

    @Enumerated(EnumType.STRING)
    private StatusVeiculo status = StatusVeiculo.DISPONIVEL;

    public enum TipoVeiculo {
        CAMINHAO_PEQUENO, CAMINHAO_MEDIO, CAMINHAO_GRANDE, CARRETA, VAN
    }

    public enum StatusVeiculo {
        DISPONIVEL, EM_VIAGEM, MANUTENCAO
    }

    public Veiculo() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getPlaca() { return placa; }
    public void setPlaca(String placa) { this.placa = placa; }

    public String getModelo() { return modelo; }
    public void setModelo(String modelo) { this.modelo = modelo; }

    public String getMarca() { return marca; }
    public void setMarca(String marca) { this.marca = marca; }

    public int getAnoFabricacao() { return anoFabricacao; }
    public void setAnoFabricacao(int anoFabricacao) { this.anoFabricacao = anoFabricacao; }
```

```

    public double getCapacidadeKg() { return capacidadeKg; }
    public void setCapacidadeKg(double capacidadeKg) { this.capacidadeKg = capacidadeKg; }

    public TipoVeiculo getTipo() { return tipo; }
    public void setTipo(TipoVeiculo tipo) { this.tipo = tipo; }

    public StatusVeiculo getStatus() { return status; }
    public void setStatus(StatusVeiculo status) { this.status = status; }
}

```

2.5. Rota.java

```

package com.transportadora.logistica.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

@Entity
@Table(name = "rotas")
public class Rota {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    private String origem;

    @NotBlank
    private String destino;

    @Positive
    private double distanciaKm;

    private double tempoEstimadoHoras;

    private String pedagogios;

    private double custoPedagio;

    public Rota() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getOrigem() { return origem; }
    public void setOrigem(String origem) { this.origem = origem; }

    public String getDestino() { return destino; }
    public void setDestino(String destino) { this.destino = destino; }

    public double getDistanciaKm() { return distanciaKm; }
    public void setDistanciaKm(double distanciaKm) { this.distanciaKm = distanciaKm; }

    public double getTempoEstimadoHoras() { return tempoEstimadoHoras; }
    public void setTempoEstimadoHoras(double tempoEstimadoHoras) { this.tempoEstimadoHoras = tempoEstimadoHoras; }

    public String getPedagogios() { return pedagogios; }
    public void setPedagogios(String pedagogios) { this.pedagogios = pedagogios; }

    public double getCustoPedagio() { return custoPedagio; }
    public void setCustoPedagio(double custoPedagio) { this.custoPedagio = custoPedagio; }
}

```

```
}
```

2.6. Carga.java

```
package com.transportadora.logistica.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

@Entity
@Table(name = "cargas")
public class Carga {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    private String descricao;

    @Positive
    private double pesoKg;

    private double volumeM3;

    @Enumerated(EnumType.STRING)
    private TipoCarga tipo;

    private boolean fragil;

    private boolean perigosa;

    public enum TipoCarga {
        GERAL, GRANEL, REFRIGERADA, PERIGOSA, FRAGIL, VIVA
    }

    public Carga() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getDescricao() { return descricao; }
    public void setDescricao(String descricao) { this.descricao = descricao; }

    public double getPesoKg() { return pesoKg; }
    public void setPesoKg(double pesoKg) { this.pesoKg = pesoKg; }

    public double getVolumeM3() { return volumeM3; }
    public void setVolumeM3(double volumeM3) { this.volumeM3 = volumeM3; }

    public TipoCarga getTipo() { return tipo; }
    public void setTipo(TipoCarga tipo) { this.tipo = tipo; }

    public boolean isFragil() { return fragil; }
    public void setFragil(boolean fragil) { this.fragil = fragil; }

    public boolean isPerigosa() { return perigosa; }
    public void setPerigosa(boolean perigosa) { this.perigosa = perigosa; }
}
```

2.7. Viagem.java

```
package com.transportadora.logistica.model;
```



```

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "viagens")
public class Viagem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "veiculo_id")
    private Veiculo veiculo;

    @ManyToOne
    @JoinColumn(name = "rota_id")
    private Rota rota;

    @ManyToOne
    @JoinColumn(name = "carga_id")
    private Carga carga;

    @ManyToOne
    @JoinColumn(name = "motorista_id")
    private Motorista motorista;

    @ManyToOne
    @JoinColumn(name = "cliente_id")
    private Cliente cliente;

    private LocalDateTime dataPartida;
    private LocalDateTime dataChegadaPrevista;
    private LocalDateTime dataChegadaReal;

    @Enumerated(EnumType.STRING)
    private StatusViagem status = StatusViagem.PLANEJADA;

    private String observacoes;

    public enum StatusViagem {
        PLANEJADA, EM_TRANSITO, ENTREGUE, CANCELADA
    }

    public Viagem() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public Veiculo getVeiculo() { return veiculo; }
    public void setVeiculo(Veiculo veiculo) { this.veiculo = veiculo; }

    public Rota getRota() { return rota; }
    public void setRota(Rota rota) { this.rota = rota; }

    public Carga getCarga() { return carga; }
    public void setCarga(Carga carga) { this.carga = carga; }

    public Motorista getMotorista() { return motorista; }
    public void setMotorista(Motorista motorista) { this.motorista = motorista; }
}

```

```

    public Cliente getCliente() { return cliente; }
    public void setCliente(Cliente cliente) { this.cliente = cliente; }

    public LocalDateTime getDataPartida() { return dataPartida; }
    public void setDataPartida(LocalDateTime dataPartida) { this.dataPartida = dataPartida; }

    public LocalDateTime getDataChegadaPrevista() { return dataChegadaPrevista; }
    public void setDataChegadaPrevista(LocalDateTime dataChegadaPrevista) { this.dataChegadaPrevista = dataChegadaPrevista; }

    public LocalDateTime getDataChegadaReal() { return dataChegadaReal; }
    public void setDataChegadaReal(LocalDateTime dataChegadaReal) { this.dataChegadaReal = dataChegadaReal; }
; }

    public StatusViagem getStatus() { return status; }
    public void setStatus(StatusViagem status) { this.status = status; }

    public String getObservacoes() { return observacoes; }
    public void setObservacoes(String observacoes) { this.observacoes = observacoes; }
}

```

2.8. Frete.java

```

package com.transportadora.logistica.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "fretes")
public class Frete {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "viagem_id")
    private Viagem viagem;

    private double distanciaKm;
    private double pesoKg;
    private String tipoCarga;

    // Custos
    private double custoCombustivel;
    private double custoManutencao;
    private double custoPedagio;
    private double custoFixo;
    private double custoVariavel;

    // Precificacao
    private double margemLucro; // percentual (ex: 15.0 = 15%)
    private double valorTotal;

    private LocalDateTime dataCriacao = LocalDateTime.now();

    public Frete() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
}

```

```
public Viagem getViagem() { return viagem; }
public void setViagem(Viagem viagem) { this.viagem = viagem; }

public double getDistanciaKm() { return distanciaKm; }
public void setDistanciaKm(double distanciaKm) { this.distanciaKm = distanciaKm; }

public double getPesoKg() { return pesoKg; }
public void setPesoKg(double pesoKg) { this.pesoKg = pesoKg; }

public String getTipoCarga() { return tipoCarga; }
public void setTipoCarga(String tipoCarga) { this.tipoCarga = tipoCarga; }

public double getCustoCombustivel() { return custoCombustivel; }
public void setCustoCombustivel(double custoCombustivel) { this.custoCombustivel = custoCombustivel; }

public double getCustoManutencao() { return custoManutencao; }
public void setCustoManutencao(double custoManutencao) { this.custoManutencao = custoManutencao; }

public double getCustoPedagio() { return custoPedagio; }
public void setCustoPedagio(double custoPedagio) { this.custoPedagio = custoPedagio; }

public double getCustoFixo() { return custoFixo; }
public void setCustoFixo(double custoFixo) { this.custoFixo = custoFixo; }

public double getCustoVariavel() { return custoVariavel; }
public void setCustoVariavel(double custoVariavel) { this.custoVariavel = custoVariavel; }

public double getMargemLucro() { return margemLucro; }
public void setMargemLucro(double margemLucro) { this.margemLucro = margemLucro; }

public double getValorTotal() { return valorTotal; }
public void setValorTotal(double valorTotal) { this.valorTotal = valorTotal; }

public LocalDateTime getDataCriacao() { return dataCriacao; }
public void setDataCriacao(LocalDateTime dataCriacao) { this.dataCriacao = dataCriacao; }
```

```
}
```

Camada de Serviço e Controle

2.9. FreteService.java

```
package com.transportadora.logistica.service;

import com.transportadora.logistica.model.*;
import com.transportadora.logistica.repository.FreteRepository;
import org.springframework.stereotype.Service;
import java.util.List;

/**
 * Servico de precificacao de fretes (modulo do Vinicius).
 *
 * A logica de calculo (constantes de custo, multiplicadores por tipo de carga
 * e formula final) foi definida por Vinicius Eduardo da Silva (RA 20240289)
 * na entrega original em docs/precificacao-etapa1-vinicius/FreteService.java.
 *
 * Esta versao integra a logica ao Spring Boot e adiciona persistencia via JPA,
 * preservando integralmente as constantes e a formula do autor original.
 *
 * Formula: ValorTotal = (CustoFixo + CustoVariavel) x Multiplicador x (1 + Margem/100)
 */
@Service
public class FreteService {

    private final FreteRepository freteRepository;

    // Constantes de custo (simulacao)
    private static final double CUSTO_COMBUSTIVEL_POR_KM = 2.50;    // R$/km
    private static final double CUSTO_MANUTENCAO_POR_KM = 0.80;    // R$/km
    private static final double CUSTO_FIXO_VIAGEM = 150.00;        // R$ por viagem
    private static final double CUSTO_POR_KG = 0.12;               // R$/kg

    // Multiplicadores por tipo de carga
    private static final double MULT_REFRIGERADA = 1.40;
    private static final double MULT_PERIGOSA = 1.60;
    private static final double MULT_FRAGIL = 1.25;
    private static final double MULT_VIVA = 1.50;
    private static final double MULT_GRANEL = 0.90;
    private static final double MULT_GERAL = 1.00;

    public FreteService(FreteRepository freteRepository) {
        this.freteRepository = freteRepository;
    }

    public Frete calcularFrete(double distanciaKm, double pesoKg, String tipoCarga,
                               double custoPedagio, double margemLucro) {

        Frete frete = new Frete();
        frete.setDistanciaKm(distanciaKm);
        frete.setPesoKg(pesoKg);
        frete.setTipoCarga(tipoCarga);
        frete.setMargemLucro(margemLucro);

        // Custos variaveis
```

```

        double combustivel = distanciaKm * CUSTO_COMBUSTIVEL_POR_KM;
        double manutencao = distanciaKm * CUSTO_MANUTENCAO_POR_KM;
        double custoPeso = pesoKg * CUSTO_POR_KG;

        frete.setCustoCombustivel(arredondar(combustivel));
        frete.setCustoManutencao(arredondar(manutencao));
        frete.setCustoPedagio(arredondar(custoPedagio));
        frete.setCustoFixo(CUSTO_FIXO_VIAGEM);

        double custoVariavel = combustivel + manutencao + custoPeso + custoPedagio;
        frete.setCustoVariavel(arredondar(custoVariavel));

        // Multiplicador por tipo de carga
        double multiplicador = getMultiplicadorTipo(tipoCarga);

        // Custo total antes da margem
        double custoBase = (CUSTO_FIXO_VIAGEM + custoVariavel) * multiplicador;

        // Aplicar margem de lucro
        double valorFinal = custoBase * (1 + margemLucro / 100.0);
        frete.setValorTotal(arredondar(valorFinal));

        return frete;
    }

    public Frete calcularESalvar(double distanciaKm, double pesoKg, String tipoCarga,
                                double custoPedagio, double margemLucro) {
        Frete frete = calcularFrete(distanciaKm, pesoKg, tipoCarga, custoPedagio, margemLucro);
        return freteRepository.save(frete);
    }

    public List<Frete> listarTodos() {
        return freteRepository.findAll();
    }

    public void deletar(Long id) {
        freteRepository.deleteById(id);
    }

    private double getMultiplicadorTipo(String tipo) {
        if (tipo == null) return MULT_GERAL;
        return switch (tipo.toUpperCase()) {
            case "REFRIGERADA" -> MULT_REFRIGERADA;
            case "PERIGOSA" -> MULT_PERIGOSA;
            case "FRAGIL" -> MULT_FRAGIL;
            case "VIVA" -> MULT_VIVA;
            case "GRANEL" -> MULT_GRANEL;
            default -> MULT_GERAL;
        };
    }

    private double arredondar(double valor) {
        return Math.round(valor * 100.0) / 100.0;
    }
}

```

2.10. ViagemController.java

```
package com.transportadora.logistica.controller;

import com.transportadora.logistica.model.*;
import com.transportadora.logistica.repository.*;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/viagens")
public class ViagemController {

    private final ViagemRepository viagemRepository;
    private final VeiculoRepository veiculoRepository;
    private final RotaRepository rotaRepository;
    private final CargaRepository cargaRepository;
    private final MotoristaRepository motoristaRepository;
    private final ClienteRepository clienteRepository;

    public ViagemController(ViagemRepository viagemRepository,
                           VeiculoRepository veiculoRepository,
                           RotaRepository rotaRepository,
                           CargaRepository cargaRepository,
                           MotoristaRepository motoristaRepository,
                           ClienteRepository clienteRepository) {

        this.viagemRepository = viagemRepository;
        this.veiculoRepository = veiculoRepository;
        this.rotaRepository = rotaRepository;
        this.cargaRepository = cargaRepository;
        this.motoristaRepository = motoristaRepository;
        this.clienteRepository = clienteRepository;
    }

    @GetMapping
    public List<Viagem> listar() {
        return viagemRepository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Viagem> buscar(@PathVariable Long id) {
        return viagemRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<?> criar(@RequestBody Map<String, Object> dados) {
        Viagem viagem = new Viagem();

        Long veiculoId = Long.valueOf(dados.get("veiculoId").toString());
        Long rotaId = Long.valueOf(dados.get("rotaId").toString());
    }
}
```

```

        Long cargaId = Long.valueOf(dados.get("cargaId").toString());
        Long motoristaId = Long.valueOf(dados.get("motoristaId").toString());

        Veiculo veiculo = veiculoRepository.findById(veiculoId).orElse(null);
        Rota rota = rotaRepository.findById(rotaId).orElse(null);
        Carga carga = cargaRepository.findById(cargaId).orElse(null);
        Motorista motorista = motoristaRepository.findById(motoristaId).orElse(null);

        if (veiculo == null || rota == null || carga == null || motorista == null) {
            return ResponseEntity.badRequest().body(Map.of("erro", "Veiculo, rota, carga ou motorista nao encontrados"));
        }

        // Regra de negocio (modulo de Logistica - Leonardo):
        // Um veiculo so pode iniciar uma viagem se estiver disponivel
        if (veiculo.getStatus() != Veiculo.StatusVeiculo.DISPONIVEL) {
            return ResponseEntity.badRequest().body(Map.of("erro",
                "Veiculo " + veiculo.getPlaca() + " nao esta disponivel (status atual: "
                + veiculo.getStatus() + ")"));
        }

        // Regra de negocio (modulo de Logistica - Leonardo):
        // O peso da carga nao pode ultrapassar a capacidade do veiculo
        if (carga.getPesoKg() > veiculo.getCapacidadeKg()) {
            return ResponseEntity.badRequest().body(Map.of("erro",
                "Peso da carga (" + carga.getPesoKg() + " kg) excede a capacidade do veiculo "
                + veiculo.getPlaca() + " (" + veiculo.getCapacidadeKg() + " kg)"));
        }

        viagem.setVeiculo(veiculo);
        viagem.setRota(rota);
        viagem.setCarga(carga);
        viagem.setMotorista(motorista);

        if (dados.containsKey("clienteId") && dados.get("clienteId") != null) {
            Long clienteId = Long.valueOf(dados.get("clienteId").toString());
            clienteRepository.findById(clienteId).ifPresent(viagem::setCliente);
        }

        viagem.setDataPartida(LocalDate.parse(dados.get("dataPartida").toString()));

        if (dados.containsKey("dataChegadaPrevista") && dados.get("dataChegadaPrevista") != null) {
            viagem.setDataChegadaPrevista(LocalDate.parse(dados.get("dataChegadaPrevista").toString()));
        }

        viagem.setObservacoes(dados.getOrDefault("observacoes", "").toString());
        veiculo.setStatus(Veiculo.StatusVeiculo.EM_VIAGEM);
        veiculoRepository.save(veiculo);

        return ResponseEntity.ok(viagemRepository.save(viagem));
    }

    @PatchMapping("/{id}/status")
    public ResponseEntity<?> atualizarStatus(@PathVariable Long id, @RequestBody Map<String, String> dados) {
        return viagemRepository.findById(id).map(v -> {

```

```

Viagem.StatusViagem novoStatus = Viagem.StatusViagem.valueOf(dados.get("status"));
v.setStatus(novoStatus);

if (novoStatus == Viagem.StatusViagem.ENTREGUE) {
    v.setDataChegadaReal(LocalDate.now());
    if (v.getVeiculo() != null) {
        v.getVeiculo().setStatus(Veiculo.StatusVeiculo.DISPONIVEL);
        veiculoRepository.save(v.getVeiculo());
    }
} else if (novoStatus == Viagem.StatusViagem.CANCELADA && v.getVeiculo() != null) {
    v.getVeiculo().setStatus(Veiculo.StatusVeiculo.DISPONIVEL);
    veiculoRepository.save(v.getVeiculo());
}

return ResponseEntity.ok(viagemRepository.save(v));
}).orElse(ResponseEntity.notFound().build());
}

@DeleteMapping("/{id}")
public ResponseEntity<?> deletar(@PathVariable Long id) {
    return viagemRepository.findById(id).map(v -> {
        viagemRepository.delete(v);
        return ResponseEntity.ok().build();
    }).orElse(ResponseEntity.notFound().build());
}
}

```